



Lab Worksheet 4

CSE360: Computer Interfacing

Department of Computer Science and Engineering

Lab 04: Introduction to SPI protocol using its master-slave architecture for communication between two STM32 microcontrollers.

I. Topic Overview

This lab worksheet is to introduce the SPI protocol on STM32 microcontrollers, using them to transmit data as a master and receive data from the master as a slave. This involves setting up the SPI communication on the STM32 microcontrollers and writing the necessary code for data transmission between master and slave.

II. Learning Outcome

After this lab, students will be able to:

1. Understand the basic principles of SPI communication and its configuration on STM32 microcontrollers.
2. Interface two STM32 microcontrollers as master-slave and implement the protocol for data transmission.
3. Configure the master and slave nodes separately for data transmission.
4. Transmit the data over SPI to one or multiple slaves, process and display it in a human-readable format.

III. Materials

- STM32 development board (e.g., STM32F446RE)
- Jumper wires
- Breadboard

IV. STM32F446RE Pins

The board consists of 64 pins out of which 38 pins are used and depicted in the following diagram. The relevant ports, pins and registers for today's lab are described later.

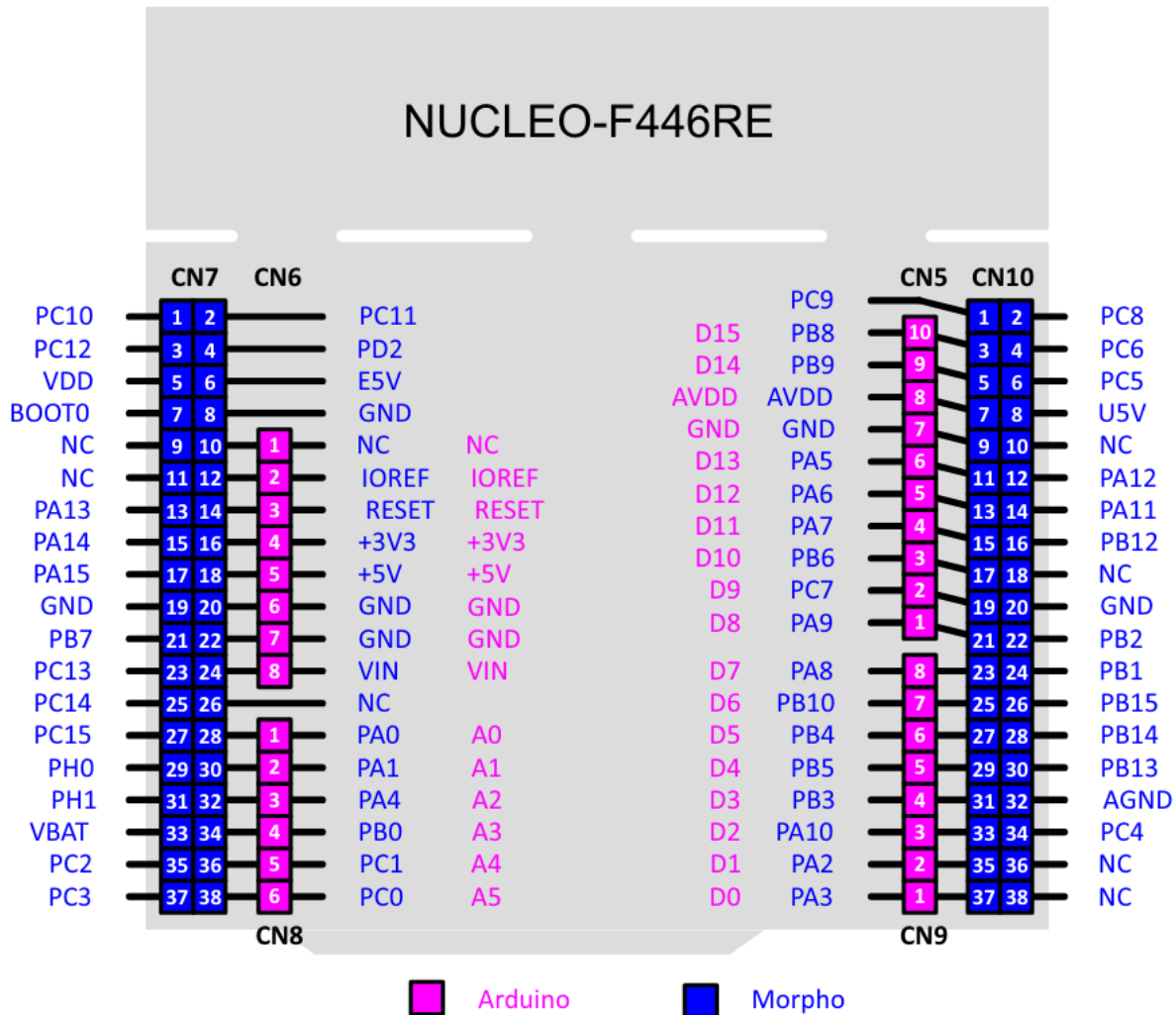


Figure 1: Pin diagram of STM32 development board

V. Registers: There are many registers related to SPI in stm32 but we only need a few of them for this lab.

Registers to Configure GPIO:

1. **32-bit AHB1 Enable register** to enable the ports. The last 8 bits are used to enable or disable the 8 ports discussed previously.

0: Port disabled

1: Port enabled

For example, to enable port A, set 1 in AHB1ENR register bit 0.

Bits 7 - 0 of RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)

7	6	5	4	3	2	1	0
GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
rw	rw	rw	rw	rw	rw	rw	rw

2. Each GPIO Port consists of a **32-bit MODER register** to configure the mode of the ports' pins. Each mode uses 2 bits for configuration. So, to configure 16 pins of a port each taking 2 bits for configuration, the 32-bit MODER register is used.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

GPIO port mode register (GPIOx_MODER) (x = A..H)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 1-0 to configure pin 1, bit 3-2 for pin 2 and so on. For example, to configure port A pin 1 in input mode, set 00 in GPIOA MODER register bit 1-0.

3. Each GPIO port includes a **32-bit OTYPER register** which defines the output type of the pins. Bit 15-0 is used to configure the output type of 16 pins.

0: Output push-pull (reset state)

1: Output open-drain

We will use the default type which is the push-pull for this lab.

4. A **32-bit GPIOx_OSPEEDR register** is used to set the output speed of each ports' pins.

00: Low speed

01: Medium speed

10: Fast speed

11: High speed

We will use high speed (11) for this lab.

GPIO port output speed register (GPIOx_OSPEEDR) (x = A..H)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

5. Each port has one **32-bit GPIOx_IDR register (read only)**, input data register where bits 15-0 are used to read the digital inputs from 16 pins.

GPIO port input data register (GPIOx_IDR) (x = A..H)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

6. Similar to the IDR register, each port has one **32-bit GPIOx_ODR register (read and write)**, output data register where bits 15-0 are used to read and write the digital outputs

of 16 pins. But we won't be writing directly in the ODR register. We will be using the GPIOx_BSRR register for atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..H).

7. Each port has a **32-bit GPIOx_BSRR register**, bit set/reset register for digital write. We can only write in this register, to read we use the ODR register.

Bit 31:16 are reset bits where,

0: No action

1: Resets corresponding ODRx pin

Bit 15:0 are set bits where,

0: No action

1: Sets corresponding ODRx pin

GPIO port bit set/reset register (GPIOx_BSRR) (x = A..H)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

8. Two registers are provided to select one out of the sixteen alternate function inputs/outputs available for each I/O. The **GPIOx_AFRL** and **GPIOx_AFRH** alternate function registers.

GPIO Alternate Function Low Register for Pin(0 to 7) (AFRL)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 AFRLy: Alternate function selection for port x bit y (y = 0..7) These bits are written by software to configure alternate function I/Os.

0000: AF0 0001: AF1 0010: AF2 0011: AF3 0100: AF4 0101: AF5 0110: AF6 0111:
AF7 1000: AF8 1001: AF9 1010: AF10 1011: AF11 1100: AF12 1101: AF13 1110: AF14
1111: AF15

[illegible]

AFRHy selection:

0000: AF0 0001: AF1 0010: AF2 0011: AF3 0100: AF4 0101: AF5 0110: AF6 0111:
AF7 1000: AF8 1001: AF9 1010: AF10 1011: AF11 1100: AF12 1101: AF13 1110: AF14
1111: AF15

- 00: No pull-up, pull-down
- 01: Pull-up
- 10: Pull-down
- 11: Reserved

[illegible]

Registers to Configure SPI:

10. We will be using SPI1, so we need to enable the peripheral clock it's connected to which is APB2. So, we will use the **RCC APB2 peripheral clock enable register (RCC_APB2ENR)** register to enable the clock for SPI1.

RCC APB2 peripheral clock enable register (RCC_APB2ENR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI2 EN	SAI1 EN	Res.	Res.	Res.	TIM11 EN	TIM10 EN	TIM9 EN
								rw	rw				rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SYSCFG EN	SPI4 EN	SPI1 EN	SDIO EN	ADC3 EN	ADC2 EN	ADC1 EN	Res.	Res.	USART6 EN	USART1 EN	Res.	Res.	TIM8 EN	TIM1 EN
	rw	rw	rw	rw	rw	rw	rw			rw	rw			rw	rw

Bit 12 SPI1EN: SPI1 clock enable

This bit is set and cleared by software.

0: SPI1 clock disabled

1: SPI1 clock enabled

11. To configure SPI communication between master-slave, we have two control registers.

The **SPI control register 1 (SPI_CR1)** and **SPI control register 2 (SPI_CR2)**.

SPI control register 1 (SPI_CR1)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	DFF	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits needed for this lab:

Bit 0 CPHA: Clock phase 0: The first clock transition is the first data capture edge 1: The second clock transition is the first data capture edge	Bit 6 SPE: SPI enable 0: Peripheral disabled 1: Peripheral enabled
	Bit 7 LSBFIRST: Frame format 0: MSB transmitted first 1: LSB transmitted first

Bit 1 CPOL: Clock polarity 0: CK to 0 when idle 1: CK to 1 when idle	Bit 8 SSI: Internal slave select This bit has an effect only when the SSM bit is set. The value of this bit is forced onto the NSS pin and the IO value of the NSS pin is ignored.
Bit 2 MSTR: Master selection 0: Slave configuration 1: Master configuration	Bit 9 SSM: Software slave management When the SSM bit is set, the NSS pin input is replaced with the value from the SSI bit. 0: Software slave management disabled 1: Software slave management enabled
Bits 5:3 BR[2:0]: Baud rate control 000: fPCLK/2 001: fPCLK/4 010: fPCLK/8 011: fPCLK/16 100: fPCLK/32 101: fPCLK/64 110: fPCLK/128 111: fPCLK/256	Bit 10 RXONLY: Receive only mode enable This bit enables simplex communication using a single unidirectional line to receive data exclusively. Keep BIDIMODE bit clear when receive only mode is active. This bit is also useful in a multi slave system in which this particular slave is not accessed, the output from the accessed slave is not corrupted. 0: full-duplex (Transmit and receive) 1: Output disabled (Receive-only mode)
	Bit 11 DFF: Data frame format 0: 8-bit data frame format is selected for transmission/reception 1: 16-bit data frame format is selected for transmission/reception

SPI control register 2 (SPI_CR2) is not needed for this lab, so the value is reset to deconfigure to remove any previous configuration.

12. To check the status of the master-slave while communicating over SPI, there is a **SPI status register (SPI_SR)**.

SPI status register (SPI_SR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	FRE	BSY	OVR	MODF	CRC ERR	UDR	CHSIDE	TXE	RXNE
							r	r	r	r	rc_w0	r	r	r	r

Bit 0 RXNE: Receive buffer not empty

0: Rx buffer empty

1: Rx buffer not empty

Bit 1 TXE: Transmit buffer empty

0: Tx buffer not empty

1: Tx buffer empty

Bit 7 BSY: Busy flag

0: SPI (or I2S) not busy

1: SPI (or I2S) is busy in communication or Tx buffer is not empty

This flag is set and cleared by hardware.

13. The data that needs to be transmitted or received is stored in the SPI **data register (SPI_DR)**.

SPI data register (SPI_DR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 15:0 DR[15:0]: Data register

Data received or to be transmitted.

The data register is split into 2 buffers - one for writing (Transmit Buffer) and another one for reading (Receive buffer). A write to the data register will write into the Tx buffer and a read from the data register will return the value held in the Rx buffer.

VI. Steps to Configure GPIO and SPI:

We have to follow some particular steps to configure GPIO and SPI to work. In STM32F446RE alternate function 5 mode, PA4 can work as SPI1_NSS pin, PA5 as SPI1_SCK, PA6 as SPI1_MISO, PA7 as SPI1_MOSI.

GPIO:

1. First we need to enable Port A using the AHB1ENR register.
2. Next we need to set the mode of PA5, PA6, PA7 to alternate mode and PA9 for master to output mode and PA4 for slave to input mode to select the slave by the master using the GPIOA_MODER register.
3. Then we need to set the mentioned pins except PA4 in the slave to high speed as SPI communication requires high speed using the GPIOA_OSPEEDR register.
4. Using the GPIOA_AFRL register, we will set PA5, PA6, PA7 to alternate function 5.

SPI:

5. To enable the SPI1 clock, we will use the APB2ENR register. Then we will disable SPI1 to configure it using the SPI1_CR1 register.
6. Using the SPI1_CR1 register we will configure SPI protocol. We are not configuring the CR2 register for this lab.
7. Lastly, after completing the configuration, we will enable SPI1 again to start the data transmission process. The data that needs to be transmitted or received is stored in the DR register.

VII. Open the Keil µVision5 IDE and follow the steps.

1. From the project tab in the menu bar, open a new project. [DO NOT put space in any file/folder names]
2. In the select device for target window, select STM32F446RETx under STMicroelectronics and click ok.

3. In the managed run-time environment window, select Core under CMSIS and Startup under Devices and click ok.
4. To create files, right click on the Source Group 1 tab under Target 1 and select Add new item. A pop up window will appear for file type. Create the following files with the mentioned type and name. Paste the following codes in their respective files.
5. From the options for target menu (magic icon), go to debug window, select ST-Link Debugger.

VII. Code

For Master: [For students boards]

- i) File name: spi, file type: C file (.c)

[spi.c](#)

For Slave: [For Faculties boards]

- i) File name: uart, file type: header file (.h)

```
#ifndef INC_UART_H
#define INC_UART_H
#include "stm32f446xx.h"
#include "stm32f446xx.h"
#include <stdio.h>

void UartInit();
void UART_SendChar(uint8_t ch);
uint8_t UART_ReceiveChar();
void UartSendBuffer(char buffer[]);
void UartReceiveBuffer(char buffer[], uint8_t len);

#endif
```

- ii) File name: uart, file type: C file (.c)

[uart.c](#)

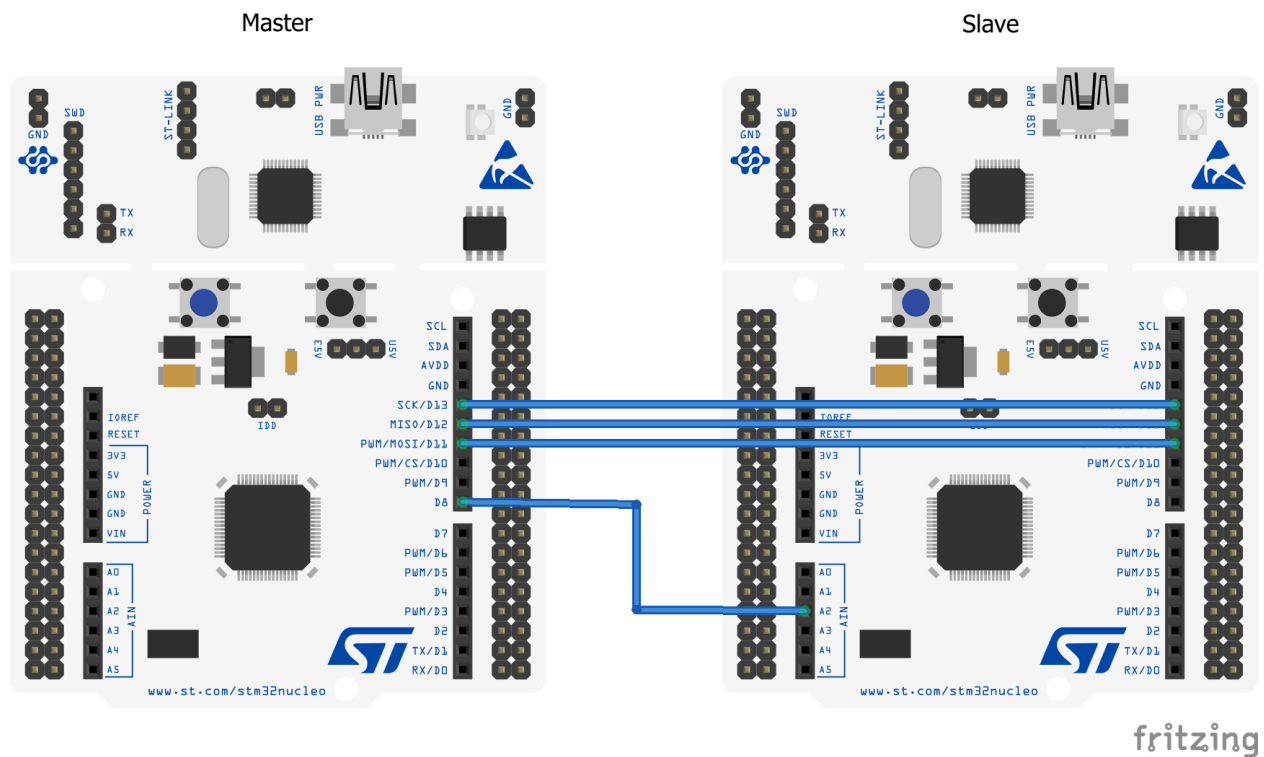
iii) File name: spi_slave, file type: C file (.c)

[spi_slave.c](#)

[Thanks to Asraful Islam Taj for his special contribution to the code segment]

After uploading the master's code in the board, connect the boards as depicted below. Both the boards then should be connected to the computer. Open hercules serial monitor and connect it to the slave's com port and open it. The monitor should start printing if everything is done correctly.

IX. Circuit Diagram



VIII. Lab evaluation:

No evaluations for this lab!

IX. References:

1. <https://www.st.com/en/microcontrollers-microprocessors/stm32f446re.html>

VII. Install hercules: It's a serial monitor. You need to install it to see the output.

Installation link: <https://www.hw-group.com/product-version/hercules> [Install the latest version]